NVIDIA

CCCL Cooperative

Hierarchy-aware groups, mappings, and synchronization

CUDA C++ Developer Walkthrough

Based on docs/cudax/coop | Experimental API



A practical tour of the group model

From the motivation to choosing the right group.

01 Model

Represent a group using its unit, boundary and hierarchy.

02 Compose

Partition with mappings; choose synchronization separately.

03 Apply

Use independent groups, virtual groups and non-owning views.

Scope: the documented C++ API in cuda::experimental::coop; algorithms are under development.

Why redesign cooperative groups?

Carry more useful structure into device-side algorithms.

More information

Expose whole warps and other hierarchy units as the building blocks of a group.

More specialization

Preserve static launch information for rank queries and resource sizing.

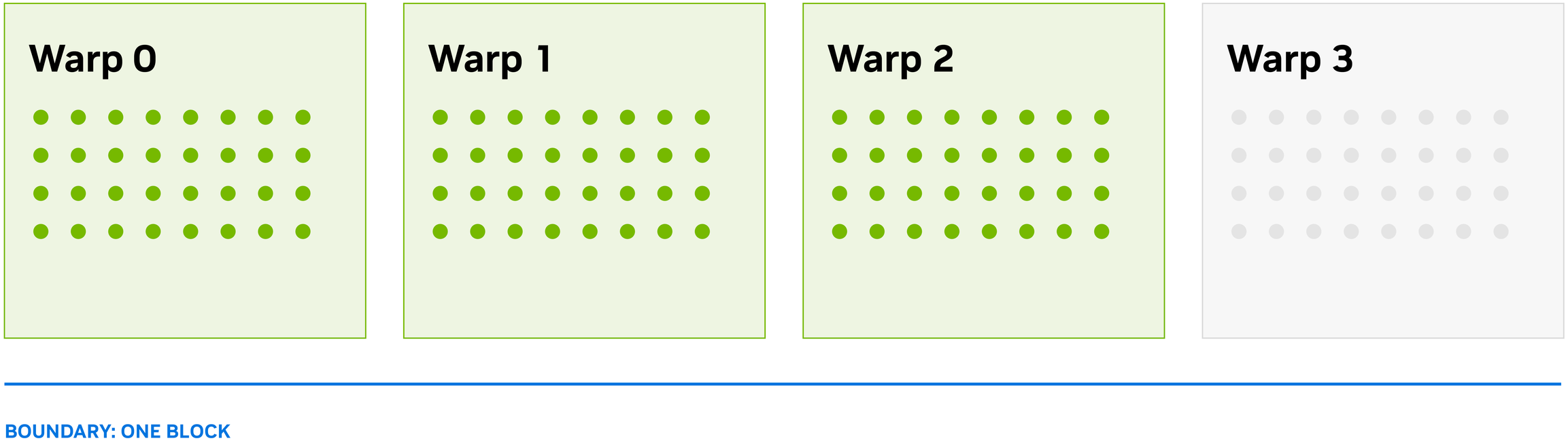
More control

Choose how to partition and how to synchronize as independent decisions.

The goal is greater flexibility, performance and safety; this deck makes no benchmark claims.

A group has a unit and an upper boundary

Example: three complete warps within one thread block.



Unit = warp · Members = 3 warps / 96 threads · Boundary = block

Launch information can simplify queries

A hierarchy-aware group can use the shape known to cuda::launch.

GENERAL 3D LINEARIZATION

```
const auto rank =
  (threadIdx.z * blockDim.y + threadIdx.y)
  * blockDim.x + threadIdx.x;
```

KNOWN 1D BLOCK

```
const auto rank = threadIdx.x;
```

Static hierarchy facts can reduce arithmetic and improve specialization opportunities.

Static group information sizes resources

Ask the group for its compile-time extent.

```
const coop::group_view view{group};
constexpr auto n = cuda::gpu_thread.static_count(view);
static_assert(n != cuda::std::dynamic_extent);
__shared__ int scratch[n];
```

- Use static_count() when the type carries the extent.
- Handle dynamic_extent when only runtime dimensions exist.

The excerpt requires a statically known extent; a runtime count cannot size a static shared array.

Four independent design choices

Define what belongs together and how it rendezvous.

What?

Choose the unit: threads, warps, blocks or another hierarchy level.

Where?

Choose the parent group and its upper boundary.

How grouped?

Choose a mapping: fixed sizes, a size list or a predicate.

How synced?

Choose a synchronizer suited to the units and scope.

docs/cudax/coop/group/index.rst

NVIDIA

The group contract is deliberately small

Types describe structure; operations express coordination.

```
using unit_type      = /* grouped unit */;
using level_type     = /* upper boundary */;
using hierarchy_type = /* launch hierarchy */;

group.hierarchy();
group.sync();
group.sync_aligned();
```

Compile-time properties

`is_always_exhaustive()`

`is_always_contiguous()`

Properties are guarantees made by the type; false means the property is not guaranteed.

Query the right level of the group

The same group can be addressed in threads or in warps.

WARP RANK 0

0 … 31

thread ranks

WARP RANK 1

32 … 63

thread ranks

WARP RANK 2

64 … 95

thread ranks

```
cuda::warp.count(g) == 3 · cuda::gpu_thread.count(g) == 96
```

Sub-unit queries: units inside the group

Read the query from left to right: unit → information → group.

```
cuda::gpu_thread.rank(g);
cuda::gpu_thread.count(g);
cuda::gpu_thread.is_root_rank(g);

cuda::warp.rank_as<cuda::std::uint32_t>(g);
cuda::warp.static_count(g);
```

Query unit must be the group unit or below it; rank is in [0, count).

Super-level queries: the group in its parent

Reverse the receiver when asking about the group itself.

```
// A child group within its parent:
child.rank(parent);
child.count(parent);

// An implicit group within a hierarchy level:
warp_group.rank(cuda::block);
warp_group.count(cuda::block);
```

Implicit groups use a native hierarchy level; child groups use their parent group.

Start from this_{level} groups

The implicit groups are the building blocks of the API.

Group	Unit = boundary	Useful query
this_thread	Thread	Thread rank in a larger level
this_warp	Warp	Thread rank inside this warp
this_block	Block	Warps / threads inside this block
this_cluster	Cluster	Blocks inside this cluster
this_grid	Grid	Blocks / threads inside this grid

Construct from a hierarchy-like object, such as the configuration received by the kernel.

docs/cudax/coop/group/this_group.rst

NVIDIA

Connect the launch to the group

The configuration carries hierarchy information into device code.

```
#define _CUDAX_ENABLE_GROUP_FEATURES_IN_LIBCUDACXX
#include <cuda/experimental/coop/group>
namespace coop = cuda::experimental::coop;

struct Kernel {
  __device__ void operator()(auto config) const {
    coop::this_block block{config};
    block.sync();
  }
};
auto config = cuda::make_config(
  cuda::grid_dims(1), cuda::block_dims<192>());
cuda::launch(stream, config, Kernel{});
```

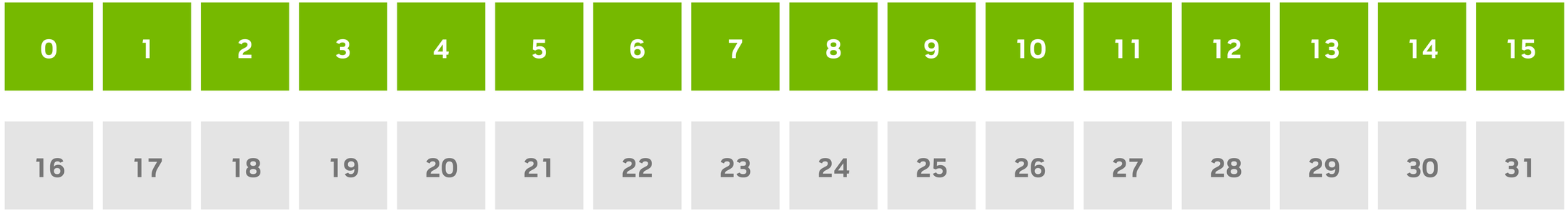

docs/cudax/coop/group/this_group.rst

NVIDIA

coalesced_group captures active lanes

Use it for opportunistic collaboration inside a warp.

```
if (cuda::gpu_thread.rank(warp) < 16) {
  coop::coalesced_group active{config};
  active.sync();
}
```



The current active set may be sparse; coalesced does not imply contiguous.

docs/cudax/coop/group/coalesced_group.rst

NVIDIA

Choose among five group forms

Ownership and synchronization independence drive the choice.

Group form	Membership	Synchronization
this_{level}	One implicit hierarchy unit	Whole level
coalesced_group	Currently active warp threads	Participating lane mask
generic_group	Mapping of a parent group	Per-subgroup synchronizer
virtual_group	Mapping of a parent group	Parent synchronization
group_view	Non-owning view of a group	Viewed group synchronization

docs/cudax/coop/group/index.rst

NVIDIA

Mappings transform membership

A mapping assigns units to groups without choosing synchronization.

Input

One mapping result describes the parent for the requested unit.

Transform

map() computes group rank, unit rank, counts and properties.

Output

The resulting membership initializes the new group.

For a unit larger than a thread, all threads in that unit must agree on assignment and participation.

docs/cudax/coop/group/mapping/index.rst

NVIDIA

A mapping result is the common currency

Queries, properties and lane masks travel together.

Information	Representative fields
Group in its parent	static_group_count(), group_count(), group_rank()
Unit in its group	static_unit_count(), unit_count(), unit_rank()
Type guarantees	is_always_exhaustive(), is_always_contiguous()
Participation	is_valid()
Warp membership	lane_mask()

If is_valid() is false, other non-static mapping-result queries are undefined behavior.

docs/cudax/coop/group/mapping/index.rst

NVIDIA

identity_mapping preserves the partition

Use a no-op mapping when generic code needs a mapping object.

```
coop::identity_mapping mapping{};
```



Same values. Same type. Same properties.

It preserves the previous mapping result instead of introducing another partition.

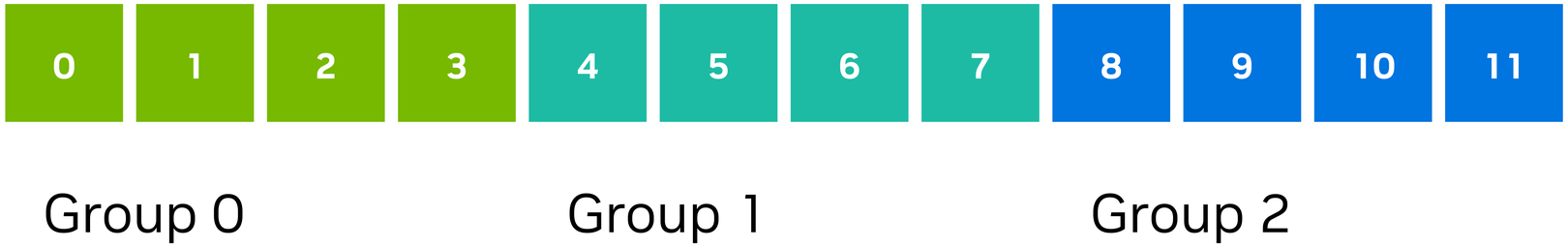
docs/cudax/coop/group/mapping/identity_mapping.rst

NVIDIA

group_by creates fixed-size groups

Split every incoming group into equally sized chunks.

```
coop::group_by dynamic_size{4};
coop::group_by static_size{cuda::std::cw<4>};
```



An integral-constant-like size preserves the group size as compile-time information.

docs/cudax/coop/group/mapping/group_by.rst

NVIDIA

Handle tails explicitly

Ten units cannot form exhaustive groups of four.



Exhaustive

Requires an exact division. A remainder violates the mapping precondition.

Non-exhaustive

Creates two full groups. The two trailing units do not participate.

A non-exhaustive mapping excludes the tail; it does not create a smaller final group.

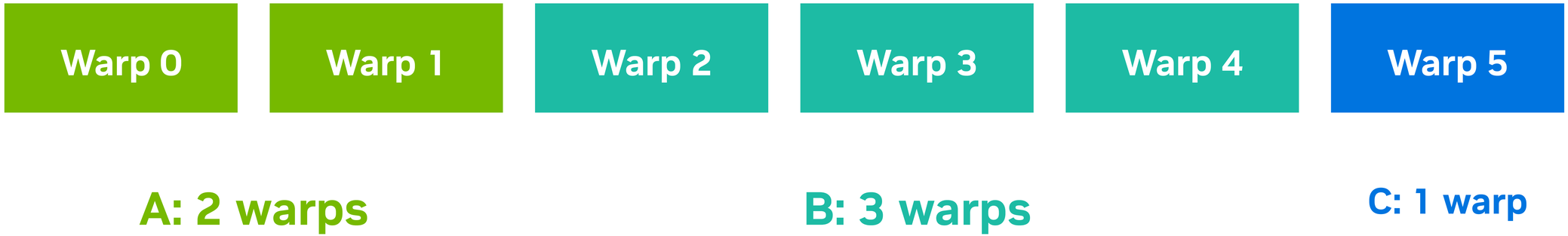
docs/cudax/coop/group/mapping/group_by.rst

NVIDIA

group_as assigns different group sizes

Give six warps to three roles: 2 + 3 + 1.

```
const cuda::std::uint32_t sizes[]{2, 3, 1};
auto mapping = coop::group_as{sizes};
```



The documented example launches 192 threads: six full warps.

docs/cudax/coop/group/generic_group.rst

NVIDIA

take selects a prefix of each group

Keep the first N units and preserve the group count.

```
coop::take first_three{3};
coop::take static_three{cuda::std::cw<3>};
```



- Contiguity is preserved when the input was contiguous.
- Participation is generally non-exhaustive.

Exhaustiveness is guaranteed only when the selected count statically equals the input count.

docs/cudax/coop/group/mapping/take.rst

NVIDIA

binary_partition splits on a predicate

A warp-local mapping can form sparse groups.

```
auto mapping = coop::binary_partition{
  [](const auto& previous) {
    return previous.unit_rank() % 2 == 1;
  }
};
```



Currently limited to threads within a warp; exhaustive, with no contiguity guarantee.

docs/cudax/coop/group/mapping/binary_partition.rst

NVIDIA

Compose mappings with the pipe operator

Transform the previous result from left to right.

```
auto mapping = coop::group_by{4} | coop::take{3};
```

FIRST: GROUP BY FOUR



THEN: RETAIN THE FIRST THREE IN EACH GROUP



docs/cudax/coop/group/mapping/composite_mapping.rst

NVIDIA

Properties must survive composition

A static guarantee must describe the physical final group.

Exhaustive

Every unit from the parent participates in a child group.

Contiguous

Members occupy consecutive physical units, ranked in ascending physical order.

Valid

This specific unit belongs to the result; later mappings preserve exclusion.

A false type property means “not guaranteed”; a runtime instance may still satisfy it.

docs/cudax/coop/group/mapping/index.rst

NVIDIA

Synchronization is a separate policy

The same membership can be paired with a suitable mechanism.

Mapping

Which units participate?
What are their ranks?



Synchronizer

How do those units rendezvous?

make_instance(unit, parent, mapping_result) prepares resources for the resulting group.

docs/cudax/coop/group/synchronizer/index.rst

NVIDIA

Match the synchronizer to the group

Unit, boundary and resource scope constrain the choice.

Synchronizer	Appropriate group	Mechanism
level_synchronizer	A whole hierarchy level	Level-wide synchronization
lane_synchronizer	Threads within one warp	__syncwarp(lane_mask)
interwarp_synchronizer	Warps within one block	Built-in named barriers
barrier_synchronizer	Group within sufficient scope	cuda::barrier objects

docs/cudax/coop/group/synchronizer/index.rst

NVIDIA

level_synchronizer follows the boundary

Whole-level synchronization underlies this_{level} groups.

Level	Synchronization behavior
Thread	No operation
Warp	__syncwarp()
Block	Named barrier 0 / __syncthreads()
Cluster	Cluster barrier arrive + wait
Grid	Block synchronization + global barrier + block synchronization

Grid sync requires device support and cuda::cooperative_launch{} in the launch configuration.

docs/cudax/coop/group/synchronizer/level_synchronizer.rst

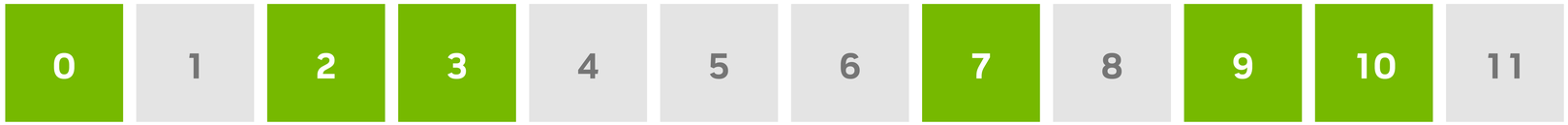
NVIDIA

lane_synchronizer uses a membership mask

Synchronize selected threads within a single warp.

```
coop::lane_synchronizer sync_policy{};

// The synchronizer uses the final mapping lane mask:
// __syncwarp(lane_mask)
```



Use for thread units inside a warp; it cannot synchronize a multi-warp subgroup.

docs/cudax/coop/group/synchronizer/lane_synchronizer.rst

NVIDIA

interwarp_synchronizer gives groups barriers

Assign distinct named barrier IDs to groups of warps.

```
coop::interwarp_synchronizer{
  cuda::std::ranges::views::iota(1)
}
```

Group A

2 warps

Barrier ID 1

Group B

3 warps

Barrier ID 2

Group C

1 warp

Barrier ID 3

IDs must be distinct and below 16; start at 1 to avoid the __syncthreads() barrier.

docs/cudax/coop/group/synchronizer/interwarp_synchronizer.rst

NVIDIA

barrier_synchronizer uses cuda::barrier

Provide storage with scope sufficient for the group.

```
__shared__ cuda::barrier<cuda::thread_scope_block>
  barriers[3];

coop::barrier_synchronizer policy{barriers};
```

- Pass a contiguous range of uninitialized barrier objects.
- Each created group receives a barrier from that range.
- Keep the storage alive for the groups that use it.

Use template argument deduction; resource count and barrier scope must cover the groups.

docs/cudax/coop/group/synchronizer/barrier_synchronizer.rst

NVIDIA

sync_aligned makes a stronger promise

All participating units execute the same synchronization statement.

sync()

Synchronize the group without promising contextual alignment.

sync_aligned()

Synchronize while guaranteeing that participants execute the exact same statement.

Both operations require valid participation and the required units to reach synchronization.

docs/cudax/coop/group/index.rst

NVIDIA

generic_group creates independent subgroups

One construction combines unit, parent, mapping and synchronizer.

```
coop::this_block block{config};
const cuda::std::uint32_t sizes[]{2, 3, 1};

coop::generic_group work{
  cuda::warp,
  block,
  coop::group_as{sizes},
  coop::interwarp_synchronizer{
    cuda::std::ranges::views::iota(1)}
};
```

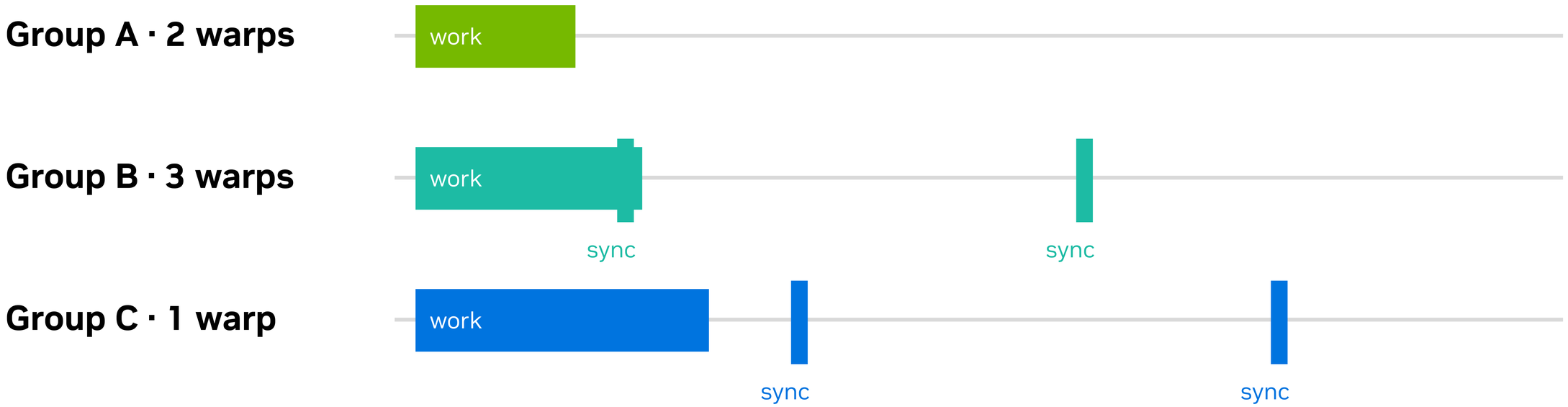
All parent units construct uniformly; after construction, subgroups synchronize independently.

docs/cudax/coop/group/generic_group.rst

NVIDIA

Independent groups can follow different schedules

Each subgroup has its own synchronization mechanism.



Different subgroups may use different counts and kinds of explicit synchronization.

docs/cudax/coop/group/generic_group.rst

NVIDIA

virtual_group reuses the parent synchronizer

Partition the work while sharing coordination resources.

```
coop::this_block block{config};
const cuda::std::uint32_t sizes[]{2, 3, 1};

coop::virtual_group work{
  cuda::warp, block, coop::group_as{sizes}
};
```

- The groups retain distinct ranks and memberships.
- Their synchronization points must match across the parent.

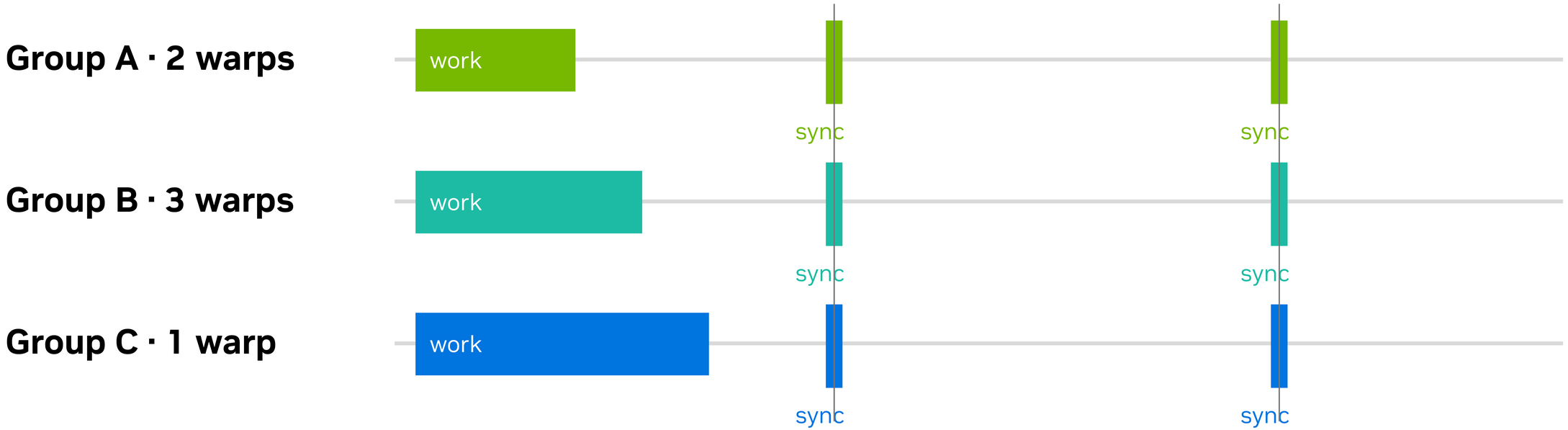
Virtual groups require exhaustive participation; an invalid mapping result is undefined behavior.

docs/cudax/coop/group/virtual_group.rst

NVIDIA

Virtual subgroups share synchronization points

Different work still has to rendezvous through the parent.



A subgroup cannot independently skip a parent-wide synchronization required by the others.

docs/cudax/coop/group/virtual_group.rst

NVIDIA

group_view adapts a group without owning it

Pass a view by value or query the same group using finer units.

```
coop::this_warp warp{config};

// View one warp as a group of threads.
coop::group_view threads{cuda::gpu_thread, warp};
const auto lane = cuda::gpu_thread.rank(threads);
threads.sync();
```

- The view keeps the original synchronization mechanism.
- The original group and its resources must outlive the view.

Owning group objects are not copyable or movable; group_view is copyable and non-owning.

docs/cudax/coop/group/group_view.rst

NVIDIA

Optional participation requires a membership check

Exclude tail units, then guard group operations.

```
coop::this_warp warp{config};
coop::generic_group selected{
  cuda::gpu_thread, warp,
  coop::group_by{coop::non_exhaustive, 10},
  coop::lane_synchronizer{}
};
if (cuda::gpu_thread.is_part_of(selected)) {
  selected.sync();
}
```

A 32-thread warp forms three 10-thread groups; its two tail threads skip group operations.

docs/cudax/coop/group/mapping/group_by.rst

NVIDIA

Choose a group by its coordination needs

Start with the synchronization contract, then compose membership.

Need	Use
An entire hierarchy level	this_{level}
The currently active subset of a warp	coalesced_group
Subgroups that synchronize independently	generic_group + compatible synchronizer
Logical subgroups with shared rendezvous	virtual_group
A by-value adapter or finer unit queries	group_view

docs/cudax/coop/group/index.rst

NVIDIA

Build groups that expose useful structure

Unit + boundary + mapping + synchronization.

- Preserve hierarchy facts so algorithms can specialize.
- Separate membership from synchronization policy.
- Use independent groups only when the synchronizer supports them.
- Check participation and respect alignment and lifetimes.

CONTINUE IN THE REPOSITORY

docs/cudax/coop · group/mapping · group/synchronizer

docs/cudax/coop/index.rst

NVIDIA